

Università della Svizzera italiana (USI)

Software Atelier 1, SA 2026-27

Final HTML project - Group 3

Abstract

This document collects the material produced by Group 3 for the final HTML project of Software Atelier 1. Section 1 presents the overall plan of the project, Section 2 contains the documentation the group uses to coordinate the work, and Section 3 is the final report with the activities performed by each team member.

Contents

1 Presentation **3**
1.1 Goals . . . 3
1.2 Team and division of work . . . 3
1.3 Timeline . . . 3

2 Quick GIT guide and CSS conventions for the project **3**
2.1 How to use Git (part 2!): branches and pull requests . . . 3
2.1.1 `main` and your other branches . . . 3
2.1.2 Step 1: get the latest version of the project . . . 4
2.1.3 Step 2: create a branch and move to it . . . 4
2.1.4 Step 3: do the work and commit it . . . 4
2.1.5 Step 4: push your branch to GitHub . . . 4
2.1.6 Step 5: open the pull request on GitHub . . . 5
2.1.7 Step 6: if changes are requested, just push again . . . 5
2.1.8 Step 7: after the merge . . . 5
2.1.9 If anything goes wrong . . . 6
2.2 Guide to the CSS . . . 6

3 Final report **6**
3.1 Team member 1 (replace with the name) . . . 6
3.2 Team member 2 (replace with the name) . . . 6
3.3 Team member 3 (replace with the name) . . . 6

1 Presentation

TODO.

1.1 Goals

TODO.

1.2 Team and division of work

TODO.

1.3 Timeline

TODO.

2 Quick GIT guide and CSS conventions for the project

2.1 How to use Git (part 2!): branches and pull requests

Our project lives in a Git repository hosted on GitHub (`https://github.com/vittodevit/sa1`). This short guide explains how we work with it: if you know how to clone, pull and push, you already know enough to follow it.

2.1.1 `main` and your other branches

The repository has one special branch, called `main`. `main` is the **real project**: the version of the website that is complete and working, the one we will hand in at the end. Nobody ever works directly on `main`.

Every time you want to modify something (a new page, a style fix, whatever), you create a **branch**: a personal copy of the project that lives next to `main` inside the same repository. While you work on your branch, the rest of the group does not see what you do, and you cannot break the real project. When your work is finished and checked, it is **merged** into `main` through a **pull request**, and from that moment it is part of the real project.

So every change always goes through the same life cycle:

- you take a fresh copy of the project by creating a branch,
- you do your work there, saving it with commits,
- you ask for your work to be merged back into `main` with a pull request on GitHub,
- after the merge the branch is not needed anymore, and your next task starts from a fresh copy again.

2.1.2 Step 1: get the latest version of the project

If it is the first time you use the repository, clone it (this downloads a full copy on your computer, and you only do it once):

```
git clone git@github.com:vittodevit/sa1.git
```

The address is the one shown by the green `Code` button on the GitHub page of the repository.

Note: what you are seeing here is SSH cloning. For this to work you need to have an SSH keypair configured on your Mac and GitHub account. I suggest setting it up by searching a tutorial because it simplifies some of the operations but token auth showed in the lectures works the same (obviousely if you clone from HTTPS)

If you already cloned it in the past, make sure you are on `main` and update it, so that you start your work from the freshest version, the one that includes what the others have merged in the meantime:

```
git checkout main
git pull
```

2.1.3 Step 2: create a branch and move to it

Before touching any file, create a branch for the task you are going to work on and switch to it. Pick a short, descriptive name, for example `home-page` or `fix-footer`:

```
git branch <branch-name>
git checkout <branch-name>
```

From this moment you are working on your copy: `main` stays safe, and you can experiment freely.

If you are not sure on which branch you currently are, `git branch` prints all the branches and marks the current one with a `*` (press 'Q' to exit).

2.1.4 Step 3: do the work and commit it

Now write your code (HTML, CSS, ...). Every time you reach a state you are happy with, save a checkpoint with a commit:

```
git add .
git commit -m "short description of what you did"
```

One commit per small working step is ideal: you will usually make one or more commits before your task is done.

2.1.5 Step 4: push your branch to GitHub

When the work is done (or when you simply want to back it up), publish your branch on GitHub with:

```
git push --set-upstream origin <branch-name>
```

The first time you push a branch you need `--set-upstream`: it tells Git to create a branch with the same name on GitHub and to connect yours to it. For all the following pushes from the same branch, a simple `git push` is enough.

2.1.6 Step 5: open the pull request on GitHub

Now go to the repository page on GitHub and ask for your work to be merged:

1. open the `Pull requests` tab at the top of the page;
2. click on the green `New pull request` button;
3. with the two selectors at the top of the page, choose `main` as the *first* one (where your work should end up) and your branch as the *second* one (what you want to merge): the page shows a summary of the changes you made;
4. click on `Create pull request`, write a meaningful title and confirm.

A pull request (PR) is simply the request "I finished something on my branch, please merge it into `main`". It is also the place where the work is checked before it becomes part of the real project.

2.1.7 Step 6: if changes are requested, just push again

Vittorio reviews every pull request. If there are issues, he will comment on the PR and ask you to modify it. In that case **do not** close the PR and **do not** create a new branch: your branch is still the right place to work in. Just make sure you are on your branch, fix what was requested, and commit and push as usual:

```
git checkout <branch-name>
git add .
git commit -m "fix the requested changes"
git push
```

The pull request updates itself automatically with the new commits, and the work continues on the same PR until it is accepted and merged.

2.1.8 Step 7: after the merge

When the PR is merged, your work is officially part of `main`. Before starting anything new, go back to `main` and pull, so that your next branch starts from a version that contains everybody's work, yours included:

```
git checkout main
git pull
```

GitHub will offer to delete the merged branch: you can safely accept, it is not needed anymore.

2.1.9 If anything goes wrong

Git can be confusing at the beginning: a commit ended up on the wrong branch, a command printed something scary, GitHub talks about *merge conflicts*, ... Do not try risky commands to "fix" things on your own: it is much easier to fix these situations early. Write your issue in the WhatsApp group of the project and describe the problem, we will fix it together.

2.2 Guide to the CSS

TODO.

3 Final report

TODO.

3.1 Team member 1 (replace with the name)

3.2 Team member 2 (replace with the name)

3.3 Team member 3 (replace with the name)